Partner Management Portal — Design & Implementation Plan

MatrixIQ · Design & Implementation Plan · Generated 2026-10-19

Status: **design only — nothing implemented yet.** Target: extend MatrixIQ in place. No separate application, no duplicate modules.

1. What MatrixIQ already is (Phase 1 analysis)

1.1 Stack

Layer	Reality
Framework	Next.js 16.3.8 (App Router), React 19, server components + route handlers
Styling	CSS Modules per component (`components/*.module.css`) + Tailwind 4 utilities; shared primitives in `components/ui/`
DB	Postgres via Sequelize 6, 76 models in `db/models/` , 99 migrations in `db/migrations/` (`sequelize-cli`)
Store layer	~130 `lib/*Store.ts` modules — **all DB access goes through these**, never from a route directly
Files	`lib/fileStorage.ts` — private blobs (Postgres `file_objects` or S3), served only via authenticated `/api/uploads/file/...`
Email	AWS SES via `lib/email/emailService.ts` + `lib/email/templates/*` , no-ops cleanly when unconfigured
In-app notifications	`lib/notificationStore.ts` (`notifyUsers`) + `lib/notificationResolver.ts` (entity_type → href)
Charts	`recharts` (currently used in `app/admin/performance-review`); most dashboards are CSS/stat-tile based
Tests	Vitest, `tests/unit/*.test.ts` , pure-logic focus with `tests/helpers/fakeSequelize.ts`

1.2 Authentication

- Single login page `app/login` → `POST /api/auth/login` → `lib/userStore.verifyLogin` → HMAC-signed token in the `nanta_session` httpOnly **session cookie** (dies on browser close, plus an idle TTL renewed by `/api/auth/heartbeat`).
- `SessionPayload` = `{ sub, username, role, exp, mustChangePassword?, isPrivileged }` . **`isPrivileged` is baked into the token at login** precisely because `proxy.ts` must never import Sequelize (documented at length in `proxy.ts`). This is the precedent the partner flag will follow.
- `proxy.ts` does coarse gating only: public paths, `mustChangePassword` lock, `/admin|/quotation-history|/api/admin` requires `isPrivileged` , `DELETE /api/admin/*` requires `superadmin` .
- Every route handler re-resolves the viewer through `lib/viewerContext.getViewerContext()` , which **re-reads the user row** (so a deactivated account dies on its next request) and resolves `isPrivileged` live from `roleStore` .

1.3 Roles & permissions

- Roles are **data**, not a union type (`UserRole = string`), seeded + reconciled in `lib/roleStore.ts`: `superadmin, admin, manager, backoffice, user (labelled "Sales"), marketing, accounts, hr` + TMS family `technical-manager, team-lead, engineer, technician`.
- `RolePermissions = { modules: Record<moduleKey, {view,create,edit,…}>, manageSettings, manageUsers, manageRoles, manageDepartments, viewAllDepartments }`.
- Three independent gates:
  1. **`isPrivileged`** — admin-area access + "sees everything" default.
  2. **`viewAllDepartments` capability** — the *only* thing that grants org-wide data visibility (`lib/departmentScope.resolveVisibilityScope`).
  3. **`isModuleActionAllowed(viewer, moduleKey, action)`** — per-module action matrix, falling back to `isPrivileged`.
- Data scoping: `resolveVisibilityScope(username)` → `{ seesOrgWide, scopedUserIds }` (own id + every member of departments they manage). `canAccessOwnedRecord`, `canAccessProject`, `canViewQuotation` / `canManageQuotation` build on it.
- **Module visibility**: `lib/moduleConfigStore.ts` seeds every nav tile (`key, label, href, section, visibleToRoles, visibleToDepartments, enabled`). `isModuleAccessAllowed()` has an escape hatch pattern already used three times — `TMS_OVERSIGHT_ROLES`, `HR_RESTRICTED_KEYS`, `ACCOUNTS_RESTRICTED_KEYS` — for modules that must **not** get the generic `isPrivileged` bypass. Partner modules need exactly this, in both directions.

1.4 Sales lifecycle (what partners plug into)

```
Lead (leads)  ──assign──►  qualification call  ──'suitable'──►  Project (projects)
                                                                       │
   stage: cold_call → catalogue_offered → site_visit → quotation → demo
          → customer_response → negotiation → po_received      (+ closed_lost)
   status: active | on_hold | won | lost
                                                                       │
   children: site_visits, quotations, demo_schedule, customer_responses,
             negotiations, purchase_orders, installations, delivery_challans,
             marketing_requests, project_notes, project_timeline_events
```

- `Project.source` is free text driven by `lib/projectSourceOptions.PROJECT_SOURCE_OPTIONS`.
- `Project.lead_confirmation_status` (`pending_confirmation|confirmed`) is an **existing overlay column** added for the Lead→Project automation — the exact precedent for a partner submission overlay.
- `Project.departments` / `department_amounts` already carry AI/AV/Robotics split.
- Quotations: `quotations` with `status draft|sent|approved|rejected`, self-referential revisions (`original_quotation_id`, `revision_number`).
- Demos: `demo_schedule` with an 11-value status enum covering technical approval → manager approval → back office → demo → DC closure.
- TMS: separate project/task system (`tms_projects`, `tms_tasks`, `tms_task_updates`) for Robotics/AI/AV/R&D, linked from `Project.tms_project_id` via `lib/tmsHandoff.ts`. **Task-oriented, not ticket-oriented — no SLA, no requester-facing comment channel.**
- Materials library: `materials` + `material_folders`, curated by Marketing (`canManageMaterials`). Every material is a Drive link, not an uploaded blob.

1.5 Gaps vs the requirement

Requirement	Exists?	Verdict
External user identity + tenant isolation	✗	**New** — biggest architectural change
Partner company master	✗	**New table**
Lead/project submission + manager review	Partial (`lead_confirmation_status`)	**Extend** `projects`
Partner-visible activity timeline	Partial (`project_timeline_events` , all internal)	**Add visibility flag**
Quotation read-only sharing	✗	**New join table**, reuse `quotations`
Demo requests	✓ `demo_schedule`	**Extend** (partner_id + derived partner status)
Marketing materials	✓ `materials`	**Extend** (publication flags + access log)
After-sales support tickets w/ SLA	✗ (TMS is task mgmt, not ticketing)	**New module** — the one genuinely new subsystem
Partner analytics	✗	**New**, reuse existing dashboard/chart conventions
Notifications	✓	**Extend** resolver with partner-aware hrefs

---

2. Core architecture decision: how a partner is isolated

This is the decision everything else hangs off. Three options were considered:

Option	Verdict
A. Separate `partner_users` table + separate auth	Rejected — breaks "same login page", duplicates session/password/audit machinery
B. Partners as ordinary `users` rows with a partner role, relying on the existing own-records-only scoping	**Rejected as the sole mechanism.** `resolveVisibilityScope` returns `[ownId]` for an unprivileged user, which *looks* safe — but every internal module would still be reachable by URL, and any record a partner created would surface in internal lists. Isolation by omission is not isolation.
C. Partners as `users` rows + a first-class `partner_id` tenancy claim enforced at three layers	**Chosen**

2.1 The three enforcement layers

Layer 1 — signed session claim (edge, no DB). Add `partnerId?: string` to `SessionPayload` in `lib/auth.ts`, computed at login/token-reissue exactly like `isPrivileged` . `proxy.ts` then gates route *areas* with zero DB access:

- Session has `partnerId` → allowed prefixes only: `/partner/**` , `/api/partner/**` , plus `CHANGE_PASSWORD_ALLOWED_PATHS` , `/profile` , `/api/profile` , `/api/auth/*` , `/api/notifications/*` , `/api/uploads*` , `/api/config/public` . Anything else → 403 (API) / redirect to `/partner` (page).
- Session has **no** `partnerId` → `/partner/**` and `/api/partner/**` are **denied** (internal staff use the internal Partner Management screens, never the partner portal).

Same staleness trade-off the file already documents and accepts for `role` / `isPrivileged`: a user's partner binding changes only on token reissue. Acceptable because partner binding never changes for a live account — suspension is handled in layer 2.

Layer 2 — viewer resolution (per request, live DB). `getViewerContext()` is the chokepoint ~every internal route already calls. Extend `ViewerContext` with `partnerId: string | null`, and — critically — **make it refuse partner accounts by default**:

```
getViewerContext(request)                       // returns null for a partner account  → 401
getViewerContext(request, { allowPartner: true })// only the partner routes opt in
getPartnerViewer(request)                       // partner routes use this; returns null for internal staff
```

This single change hardens roughly every existing internal API against partner access without editing ~35 route files. It also revalidates the partner row live, so **suspending a partner kills every in-flight session on its next request** (same mechanism that already kills deactivated users).

Layer 3 — query-level tenancy. Every partner store function takes `partnerId` as a **mandatory first argument** and puts it in the `WHERE`, never as an optional filter. No partner-facing query may be written without it. Enforced by convention + a dedicated test suite (§8).

2.2 Roles

Seed two roles in `lib/roleStore.ts` (new `PARTNER_SEED_ROLES` block, mirroring `TMS_SEED_ROLES`):

key	label	isPrivileged	viewAllDepartments	notes
`partner-admin`	Partner Admin	false	false	primary account; may create/manage `partner-user` accounts **within their own partner**, submit, request demos, raise tickets
`partner-user`	Partner User	false	false	same read surface, no sub-user management

Both are `isSystem: true`. **Neither is added to `ALL_ROLES` in `moduleConfigStore.ts`** — adding them there would silently grant every Sales module. Add `ROLE_LABEL` entries in `components/Sidebar.tsx`.

Partner self-registration does not exist: there is no public signup route, and partner `users` rows can only be created by `/api/admin/partners/*` (internal, permission-checked).

---

3. Data model

3.1 New tables

`partners` — the company master.

```
id uuid pk
partner_code        varchar(32)  unique not null      -- human reference, e.g. PTR-0007
company_name        varchar(255) not null
contact_person      varchar(255)
email, phone        varchar
address, city, state, country  varchar
gst_number          varchar(32)
```

```
status              varchar(20) not null default 'pending'
                    -- pending | active | suspended | inactive
departments         jsonb  default '[]'   -- reuses lib/projectDepartmentOptions values (ai|av|robotics)
product_categories  jsonb  default '[]'
territory           jsonb  default '[]'
relationship_manager_id uuid → users(id)  ON DELETE SET NULL   -- internal RM
notes               text
created_by, updated_by uuid → users(id)
created_at/updated_at/deleted_at           -- paranoid, matching every other model
```

Indexes: `partner_code` (unique), `status`, `relationship_manager_id`.

`partner_invites` — activation/recovery tokens.

```
id, partner_id → partners, user_id → users, token_hash varchar(128) not null,
purpose varchar(20) ('activation'|'recovery'), expires_at, used_at, created_by, created_at
```

Only a SHA-256 hash is stored; the raw token lives in the emailed link only. Single-use, default 72h.

`partner_quotation_shares` — explicit publish of a quotation to a partner.

```
id, quotation_id → quotations, partner_id → partners,
shared_by uuid → users, shared_at, revoked_at, note text
unique (quotation_id, partner_id) where revoked_at is null
```

Absence of a row = invisible. A draft quotation can never be shared (enforced in the store, not the schema).

`support_tickets` — the one genuinely new subsystem.

```
id, ticket_number varchar(32) unique not null       -- TKT-2026-0001, same generator style as
lib/quotationNumber.ts
partner_id → partners (nullable — internal staff may raise tickets too)
created_by_user_id → users
project_id → projects (nullable), quotation_id → quotations (nullable), installation_id (nullable)
title varchar(255) not null
category varchar(40)         -- issue category
issue_type varchar(40)       --
technical|installation|configuration|maintenance|warranty|amc|product_failure|other
product varchar(255), model_serial varchar(120)
customer_name varchar(255), location varchar(255)
description text not null
priority varchar(10)         -- low|medium|high|critical
status varchar(30) not null default 'new'
department_id → departments (nullable)   -- routing queue
assigned_to_id → users (nullable), assigned_by_id, assigned_at, acknowledged_at
occurred_on dateonly
first_response_at, resolved_at, closed_at, reopened_count int default 0
response_due_at, resolution_due_at       -- stamped from the SLA policy at triage
resolution_summary text, root_cause text, corrective_action text
attachments jsonb default '[]'           -- matches the app-wide inline-JSONB attachment convention
contact_person varchar(255), contact_method varchar(40)
created_at/updated_at/deleted_at (paranoid)
```

Indexes: `(partner_id, status)`, `(assigned_to_id, status)`, `status`, `priority`, `department_id`, `project_id`, `resolution_due_at`.

`support_ticket_comments`

```
id, ticket_id → support_tickets, author_user_id → users,
body text not null,
is_internal boolean not null default true,   -- DEFAULT INTERNAL. Partner-visible is the deliberate act.
attachments jsonb default '[]', created_at
```

Index `(ticket_id, created_at)`.

`support_ticket_status_history`

```
id, ticket_id, from_status, to_status, by_user_id, reason text, at
```

`support_sla_policies`

```
id, name, priority varchar(10) unique,
first_response_minutes int, resolution_minutes int,
pause_on_awaiting_partner boolean not null default true,
business_hours_only boolean not null default false,
escalate_to_user_id → users (nullable), escalate_after_minutes int,
status varchar(20) default 'active'
```

Seeded: critical 30m/4h, high 2h/8h, medium 4h/24h, low 8h/72h — all admin-editable.

`material_access_logs` (download/view tracking)

```
id, material_id → materials, user_id → users, partner_id (nullable), action varchar(10), at
```

3.2 Columns added to existing tables

Table	Column	Why
`users`	`partner_id uuid → partners ON DELETE RESTRICT` (null for every internal user)	the tenancy claim. Index it.
`projects`	`partner_id uuid → partners`	partner-sourced attribution
`projects`	`partner_submission_status varchar(30)` — `submitted | info_requested | approved | rejected`, NULL on every normal project	**mirrors the existing `lead_confirmation_status` overlay exactly**
`projects`	`partner_reviewed_by uuid`, `partner_reviewed_at`, `partner_review_remarks text`	review audit
`projects`	`partner_submitted_value decimal(14,2)`, `partner_expected_timeline varchar(120)`	partner's own estimate, kept **separate** from `approx_price` so an unvetted number never pollutes the sales forecast
`project_notes`	`partner_visible boolean not null default false`	internal notes stay internal

Table	Column	Why
`project_timeline_events`	`partner_visible boolean not null default false`	ditto
`demo_schedule`	`partner_id uuid`, `requested_by_partner_user_id uuid`	partner-requested demos
`demo_schedule`	`partner_remark text`	the partner-facing status note (internal notes stay in `notes`)
`materials`	`partner_visible boolean not null default false`, `partner_audience jsonb default '[]'` (`[]` = all partners when `partner_visible`)	publication control, **default closed**
`material_folders`	`partner_visible boolean not null default false`	a folder is invisible unless published; a published material inside an unpublished folder is still hidden
`quotations`	*(none)*	sharing lives entirely in `partner_quotation_shares`

`PROJECT_SOURCE_OPTIONS` gains `'Partner'` (append before `'Other'`).

3.3 Migrations (in order)

```
20261020090000-create-partners.js
20261020091000-users-partner-id.js
20261020092000-create-partner-invites.js
20261020093000-projects-partner-submission.js
20261020094000-project-notes-timeline-partner-visible.js
20261020095000-demo-schedule-partner-fields.js
20261020096000-create-partner-quotation-shares.js
20261020097000-materials-partner-publication.js
20261020098000-create-material-access-logs.js
20261020100000-create-support-tickets.js
20261020101000-create-support-ticket-comments-history.js
20261020102000-create-support-sla-policies.js
```

Conventions to follow (per `20261009190000-create-material-folders.js`): `gen_random_uuid()` defaults, explicit `onUpdate/onDelete`, named indexes, snake_case columns, `created_at/updated_at/deleted_at`, a real `down()`.

> ⚠️ Per standing project rule: **no migration is run without explicit approval.** Note also that the DB currently carries orphaned `daily_task_*` tables ahead of the repo — confirm migration state before adding to the chain.

---

4. Backend: stores and APIs

4.1 New `lib/` modules

File	Responsibility
`partnerStore.ts`	CRUD on `partners`, code generation, status transitions
`partnerUserStore.ts`	create/suspend partner logins, bind `users.partner_id`
`partnerInviteStore.ts`	token issue/verify/consume (hash-only)
`partnerAccess.ts`	`canManagePartners(viewer)`, `canReviewPartnerSubmission(viewer, partnerId)`, `assertPartnerOwns(partnerId, record)`
`partnerContext.ts`	`getPartnerViewer(request)` → `{ userId, username, name, role, partnerId, partnerStatus }` or null
`partnerProjectStore.ts`	submission create/list/review, all partner-scoped
`partnerProjectStages.ts`	**pure** — internal `ProjectStage` + `partner_submission_status` → the 11 partner-facing stages. No new DB enum values.
`partnerQuotationShareStore.ts`	share/revoke/list; refuses drafts
`partnerDemoStatus.ts`	**pure** — `demo_schedule.status` → partner-facing demo status
`partnerAnalytics.ts`	partner KPI aggregation (one partner, or consolidated for internal)
`partnerVisibility.ts`	what a partner may read on a project / demo / quotation — the single redaction point
`supportTicketStore.ts`	ticket CRUD, assignment, comments, attachments
`supportTicketWorkflow.ts`	**pure** — the role × status transition matrix
`supportSla.ts`	**pure** — due-date computation, pause rules, breach/overdue classification
`supportTicketAccess.ts`	who may read/assign/comment/close
`materialPublication.ts`	partner-visible material/folder resolution + access logging
`email/templates/partnerInvite.ts`, `partnerLifecycle.ts`, `supportTicketLifecycle.ts`	SES templates matching the existing layout helpers

Pure libs (`partnerProjectStages`, `partnerDemoStatus`, `supportTicketWorkflow`, `supportSla`, `partnerVisibility`) stay import-free of Sequelize so they are directly unit-testable — the same split already used by the Daily Report module.

4.2 Internal APIs (staff)

```
GET    /api/admin/partners                  list + filters
POST   /api/admin/partners                  create partner (+ optional first login)
GET    /api/admin/partners/[id]             profile + rollup counts
PATCH  /api/admin/partners/[id]             edit, assign RM, activate/suspend/reactivate
POST   /api/admin/partners/[id]/users       create a partner login
```

```
PATCH  /api/admin/partners/[id]/users/[uid]    suspend / reset access
POST   /api/admin/partners/[id]/invite         (re)send activation link
GET    /api/admin/partners/analytics           consolidated partner performance

GET    /api/projects?source=partner            existing route, new filter
POST   /api/projects/[id]/partner-review       approve | reject | request-info  (Sales Manager)
POST   /api/projects/[id]/notes                existing — gains `partnerVisible` flag

POST   /api/quotations/[id]/share-partner      publish a non-draft quotation
DELETE /api/quotations/[id]/share-partner      revoke

GET    /api/support/tickets                    internal queues (unassigned/mine/overdue/escalated/…)
GET    /api/support/tickets/[id]
PATCH  /api/support/tickets/[id]               triage, priority, status transition
POST   /api/support/tickets/[id]/assign
POST   /api/support/tickets/[id]/comments      { body, isInternal }
POST   /api/support/tickets/[id]/escalate
GET/PUT /api/admin/support/sla                 SLA policy config

PATCH  /api/materials/[id]                     existing — gains partner publication fields
```

4.3 Partner APIs (all under one prefix, all tenancy-scoped)

```
GET  /api/partner/me                     partner profile + account state
GET  /api/partner/dashboard              KPI summary
GET  /api/partner/projects               own submissions only
POST /api/partner/projects               submit a lead/project
GET  /api/partner/projects/[id]          redacted detail + partner-visible timeline
POST /api/partner/projects/[id]/info     respond to an info request
GET  /api/partner/quotations             shared, non-draft only
GET  /api/partner/quotations/[id]/pdf    reuses lib/quotationPdfFromRecord.ts
GET/POST /api/partner/demos              list / request
POST /api/partner/demos/[id]/reschedule  request only — never direct assignment
GET  /api/partner/materials              published tree only (+ access log on open)
GET/POST /api/partner/support/tickets
GET  /api/partner/support/tickets/[id]
POST /api/partner/support/tickets/[id]/comments     is_internal forced false
POST /api/partner/support/tickets/[id]/confirm      confirm resolution / request reopen
GET  /api/partner/analytics
GET  /api/partner/notifications          reuses notificationStore, partner-aware hrefs
```

Rules applied uniformly: resolve viewer via `getPartnerViewer`; reject if `partner.status !== 'active'`; `partnerId` in every `WHERE`; responses built by `partnerVisibility.ts` (allow-list of fields, never `delete` on an internal record).

---

5. Frontend

5.1 Internal (extends existing nav)

Route	Component	Section
`/admin/partners`, `/admin/partners/[id]`	`PartnersView`, `PartnerDetailView`	Sales
`/projects`	existing `ProjectsView` — add a **Source** filter (Partner-Sourced / Direct / all) and a "Partner" badge column showing partner name + code	Sales
`/projects/[id]`	existing `ProjectDetailView` — add a Partner panel, the review action bar for pending submissions, and a `partner-visible` toggle on each note	Sales
`/admin/partners/analytics`	`PartnerAnalyticsView` (recharts)	Sales
`/support`, `/support/[id]`	`SupportTicketsView`, `SupportTicketDetailView`	Support (new section)
`/admin/support/sla`	`SupportSlaConfigView`	Admin
`/materials`	existing `MaterialsView` — publication toggles for Marketing	Marketing

5.2 Partner portal (`app/partner/**`)

`PartnerShell` wraps `AppShell` so the chrome, session watcher, header and CSS-module design system are literally the same — only the nav set differs.

```
/partner                    Dashboard (KPI tiles + recent activity)
/partner/projects           My Leads / Projects  (+ /new, /[id])
/partner/projects/[id]      Progress stepper + partner-visible timeline
/partner/quotations         shared quotations (read-only, download)
/partner/demos              demo requests (+ /new)
/partner/support            tickets (+ /new, /[id])
/partner/materials          published library
/partner/notifications
/partner/profile
```

Every page is also guarded server-side by `lib/partnerPageGuard.ts` — `requirePartnerPage()`, modelled exactly on `lib/tmsPageGuard.ts` (cookie → session → user → partner → redirect), because `proxy.ts` cannot touch the DB.

5.3 Module registry & sidebar

Add to `SEED_MODULES` a `Partner` section: `partner-dashboard, partner-projects, partner-quotations, partner-demos, partner-support, partner-materials` with `visibleToRoles: ['partner-admin','partner-user']`; plus internal `partners` and `support-tickets` entries.

Then add two new escape-hatch sets in `isModuleAccessAllowed`, following the `HR_RESTRICTED_KEYS` / `ACCOUNTS_RESTRICTED_KEYS` precedent:

```
const PARTNER_PORTAL_KEYS = new Set([...]);   // partner roles ONLY — no isPrivileged bypass
const PARTNER_ROLE_KEYS   = ['partner-admin','partner-user'];
```

Without this, every `manager` account would see the partner portal tiles (the generic `isPrivileged` bypass), and partner roles would be unable to see them at all (they are not in `ALL_ROLES`). Both directions must be handled.

5.4 Login redirect

`app/login` currently routes to `/` on success. Add: if the login response says the account is a partner, push `/partner`. `app/page.tsx` also redirects partner sessions to `/partner`, so a bookmarked `/` can't land a partner on the internal dashboard.

---

6. Workflows

6.1 Partner submission → sales

```
partner submits  →  projects row: source='Partner', partner_id set,
                    partner_submission_status='submitted', stage='cold_call',
                    created_by = the partner's RM (so internal scoping works),
                    timeline event 'partner_submitted' (partner_visible=true)
       ↓ notify RM + Sales Managers of the partner's departments
Sales Manager: approve | request info | reject(reason)
       ↓ approve
partner_submission_status='approved'; assign salesperson via the existing
lib/projectSalesOwner.ts path (sets created_by + sales_person, notifies)
       ↓
project behaves as an ordinary MatrixIQ project from here on — same id, same
stage machine, same children. Partner badge persists forever.
```

Projects with `partner_submission_status IN ('submitted','info_requested')` are **excluded from pipeline KPIs and forecast totals** until approved — they are not yet qualified business.

6.2 Partner-facing stage mapping (no new DB values)

Partner sees	Derived from
Submitted / Awaiting Sales Review	`partner_submission_status = submitted`
Information Requested	`= info_requested`
Rejected	`= rejected`
Approved → Assigned to Salesperson	`= approved` + `sales_person` set
Requirement Analysis	stage `cold_call` / `catalogue_offered` / `site_visit`
Demo Requested / Scheduled / Completed	linked `demo_schedule` status
Quotation Shared	a live `partner_quotation_shares` row exists
Negotiation	stage `negotiation`
Won / Lost / On Hold	`status`

6.3 Support ticket lifecycle

```
new → under_triage → assigned → acknowledged → in_progress
        ↕ awaiting_partner_info / awaiting_internal / escalated
      → resolved → (partner confirms) closed
                 → (partner reopens within N days) reopened → assigned
      → cancelled (manager/admin only)
```

Transitions are a role × from → to matrix in `supportTicketWorkflow.ts`:

- Partner may only: create, comment, `awaiting_partner_info → in_progress` (by replying), `resolved → closed` (confirm) or `resolved → reopened`.
- Engineer: acknowledge, in_progress, awaiting_*, resolved, escalate.
- Technical Manager: triage, priority, assign/reassign, cancel, force-close.
- Invalid transitions are rejected server-side with 409, and every accepted one writes `support_ticket_status_history`.

SLA: `response_due_at` / `resolution_due_at` stamped at triage from the priority policy; clock pauses while `awaiting_partner_info` when the policy says so. Overdue/breach is **computed, not stored**, so changing a policy re-evaluates existing tickets consistently.

6.4 Demo requests

Partner request creates a `demo_schedule` row with `status='draft'`, `partner_id`, `created_by` = the assigned salesperson (so the existing scoping and approval chain work untouched). From there it runs the existing technical-approval → manager-approval → back-office flow with **zero changes**. The partner sees the derived status only, and can never set `assigned_technical_person_id`.

---

7. Permission matrix (reconciled with the real model)

Legend: ✓ allowed · ✗ denied · ⊙ scoped (own partner / own records / managed department)

Capability	Partner Admin	Partner User	Salesperson (`user`)	Sales Manager (`manager`/dept mgr)	Engineer	Technical Manager	Admin/Superadmin
Create/manage partner accounts	✗	✗	✗	✓	✗	✗	✓
Manage own sub-users	✓ ⊙	✗	–	–	–	–	✓
Submit lead/project	✓ ⊙	✓ ⊙	✓	✓	✓ (for a sales owner)	✗	✓
Review/approve partner submission	✗	✗	✗	✓ ⊙	✗	✗	✓
Assign salesperson	✗	✗	✗	✓	✗	✗	✓
View project detail	⊙ redacted	⊙ redacted	⊙	⊙	⊙ assigned	⊙	✓

Capability	Partner Admin	Partner User	Salesperson (`user`)	Sales Manager (`manager`/dept mgr)	Engineer	Technical Manager	Admin/Superadmin
Internal notes	✗	✗	✓	✓	✓	✓	✓
Create/edit quotation	✗	✗	✓	✓	⊙ on-behalf	✗	✓
View quotation	shared only	shared only	⊙	⊙	⊙ via project	⊙	✓
Share quotation with partner	✗	✗	✓ ⊙	✓	✗	✗	✓
Request demo	✓ ⊙	✓ ⊙	✓	✓	✗	✗	✓
Assign technical staff to demo	✗	✗	✓	✓	✗	✓	✓
Create support ticket	✓ ⊙	✓ ⊙	✓	✓	✓	✓	✓
Assign ticket	✗	✗	✗	✗	✗	✓	✓
Internal ticket notes	✗	✗	⊙	⊙	✓	✓	✓
Resolve ticket	✗	✗	✗	✗	✓ assigned	✓	✓
Confirm/reopen resolution	✓ ⊙	✓ ⊙	–	–	–	–	✓
Analytics	⊙ own partner	⊙ own partner	⊙	⊙ + partner comparison	⊙ technical	⊙ team	✓
Marketing materials	published only	published only	all internal	all internal	all internal	all internal	✓ + publish

Existing stricter rules win wherever they conflict — nothing in this table loosens a current restriction.

---

8. Testing

Unit (Vitest, the pure libs — this is where the real logic lives): `partnerProjectStages`, `partnerDemoStatus`, `supportTicketWorkflow` (full transition matrix incl. every rejected transition), `supportSla` (due dates, pause windows, breach classification), `partnerVisibility` (field allow-lists), `materialPublication`, partner code generation, invite token hash/expiry.

Isolation suite — `tests/unit/partnerIsolation.test.ts`, mandatory: Partner A must get 404/403 (never 200, never an empty-but-leaky payload) for Partner B's project, quotation, demo, ticket, ticket comment, attachment, analytics, and material. Plus: a partner hitting any `/api/*` route outside `/api/partner/**`; an internal user hitting `/api/partner/**`; a suspended partner on every route; a partner attempting `assigned_technical_person_id`, quotation mutation, or `is_internal: true` on a comment.

Regression: existing login, project create/stage change, quotation create + PDF, demo approval chain, TMS handoff, materials, dashboards, sidebar for all 13 existing roles.

Security review: run `/security-review` on the branch before merge. Specifically verify attachment downloads (`/api/uploads/file/...`) are partner-scoped — today that route authenticates the session but does not check record ownership, which is fine for internal staff and **not** fine for partners.

> Open item flagged now, not silently assumed: attachment authorization needs a per-record ownership check added for partner sessions. Scoped into Phase 4.

---

9. Delivery phases

Phase	Scope	Depends on
P0 — Foundations	`partners` + `users.partner_id` migrations, partner roles, `SessionPayload.partnerId` , `proxy.ts` prefix gating, `getViewerContext({allowPartner})` , `getPartnerViewer` , `partnerPageGuard` , module registry + sidebar, login redirect, empty `/partner` shell	—
P1 — Partner management	Internal partner CRUD, sub-users, invites + SES email, status lifecycle, relationship manager, partner profile page	P0
P2 — Submission & sales integration	Project overlay columns, partner submission form, review/assign workflow, Projects list source filter + partner badge, partner-visible timeline flags	P1
P3 — Progress, quotations, demos	Partner project progress page, quotation sharing + read-only view/PDF, demo request + derived status, attachment scoping fix	P2
P4 — Analytics & materials	Partner analytics dashboard, internal consolidated comparison, material publication flags + access logs	P2
P5 — Support	Ticket tables, internal support dashboard + triage/assignment/engineer workflow, SLA engine + config, partner support portal	P1
P6 — Notifications, hardening, docs	All notification hooks + partner-aware resolver hrefs, isolation + regression suites, security review, API/permission documentation	P2–P5

Each phase ships behind its Module Manager entries disabled by default, so a half-finished phase is invisible in production.

---

10. Risks & decisions to confirm before coding

1. **Migration approval.** Twelve migrations touch `users` and `projects` . Per project rule these need explicit sign-off, and the DB is already ahead of the repo (orphaned `daily_task_*` tables) — reconcile migration state first.
2. **`created_by` on partner submissions.** Setting it to the relationship manager keeps every existing scoping query working unchanged. The alternative (the partner's own user id) would make partner rows appear in `resolveVisibilityScope` results for whoever manages that "department" — messy. **Recommendation: RM.** Needs confirmation because it changes who sees an unreviewed submission.
3. **Sales Manager identification.** There is no `sales-manager` role — the app resolves it from `Department.managerIds` for `['Sales','GEM - Sales']` (`isSalesLeadManager`). Partner review authority should reuse that function rather than inventing a role.

4. **Session staleness.** `partnerId` in the token won't change until reissue. Suspension is handled live in `getViewerContext`; partner *rebinding* would require a forced logout. Acceptable, documented.
5. **Demo status vocabulary.** The spec lists 10 demo statuses; the DB enum has 11 different ones. Mapping (not extending the enum) avoids a Postgres enum migration and keeps the back-office DC flow intact.
6. **TMS vs. Support.** Deliberately **not** reusing `tms_tasks` for tickets: no SLA, no requester channel, no internal/external comment split, and its status enum is task-shaped. A ticket may *spawn* a TMS task later — that link is out of scope for v1.
7. **Notification hrefs.** `notificationResolver.ts` returns internal routes. Partner recipients need `/partner/...` equivalents, so the resolver must become viewer-aware. Small but easy to miss.
8. **Materials are Drive links.** Partner "download" opens a Google Drive URL whose sharing is controlled in Drive, not MatrixIQ. If partner materials must be access-controlled end-to-end, they need to move to `fileStorage.ts` blobs — a scope increase worth deciding on now.